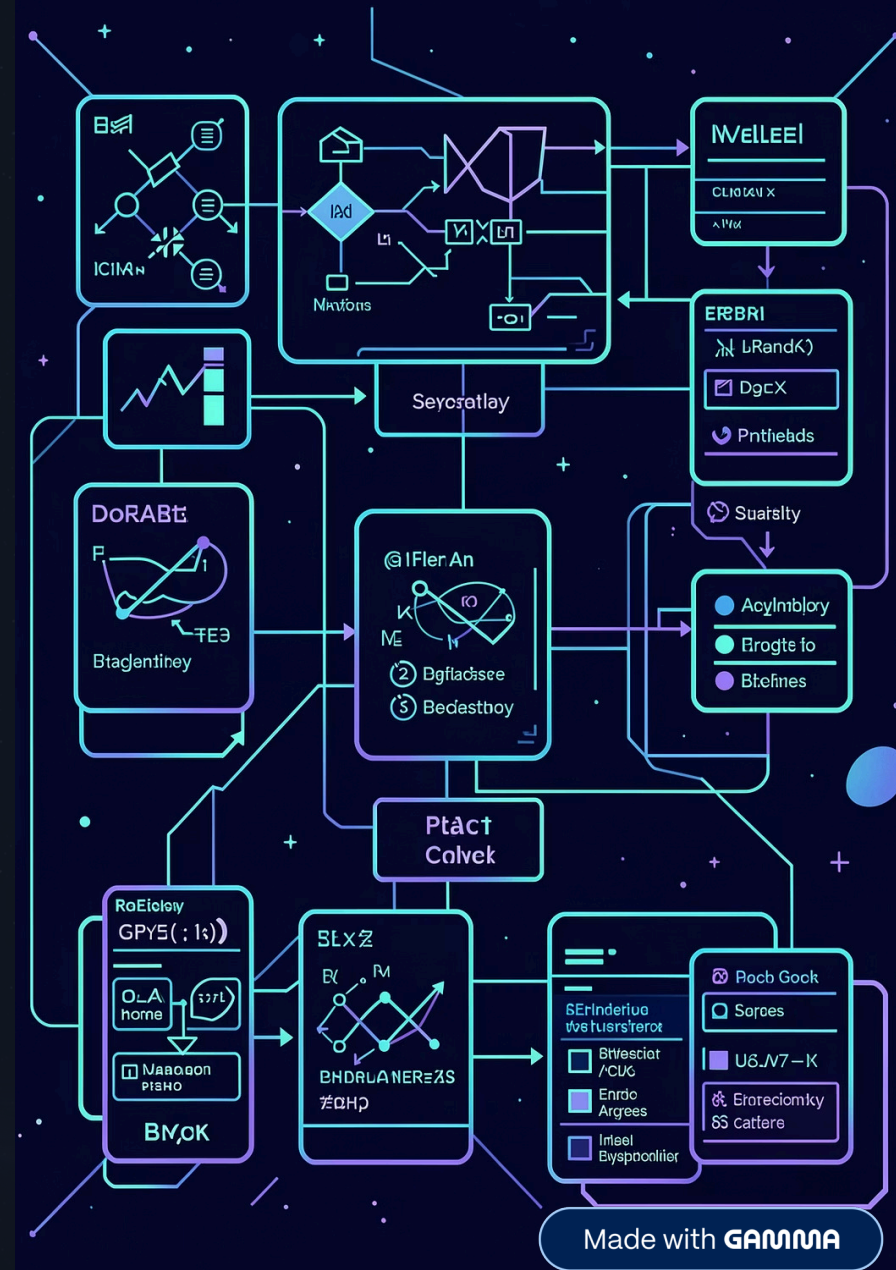


Selección de Estructuras de Datos según el Problema

Asignatura: Desarrollo de Software



¿Por qué la selección de estructuras de datos es fundamental?

Relación problema-estructura

La naturaleza del problema determina la estructura de datos más adecuada. Cada problema tiene características específicas que deben ser analizadas antes de seleccionar la estructura.

Impacto en rendimiento

La elección incorrecta puede afectar significativamente la eficiencia del programa en tiempo de ejecución y uso de memoria.

Criterios fundamentales para la selección



Tipo de acceso

Acceso directo (índice) vs. acceso secuencial



Volumen de datos

Cantidad esperada de elementos a almacenar



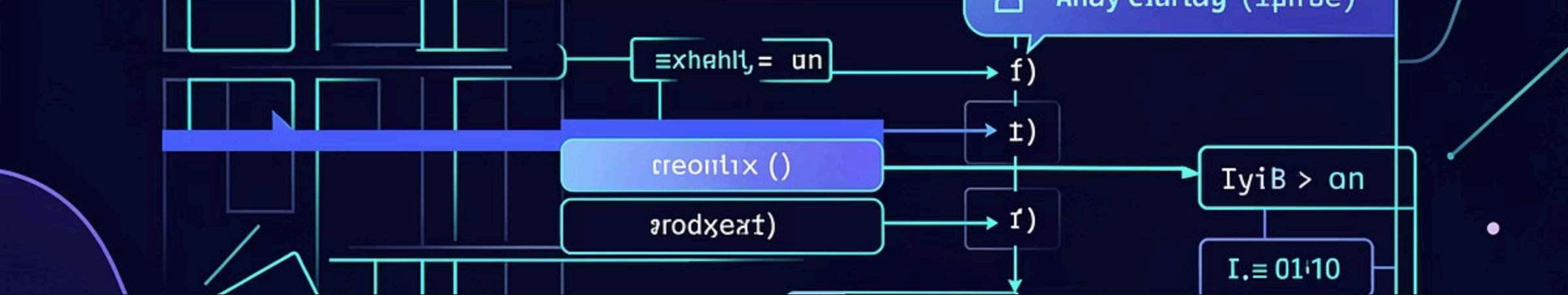
Operaciones frecuentes

Inserción, eliminación, búsqueda, modificación



Gestión de memoria

Memoria estática vs. dinámica, fragmentación



Arrays: estructuras lineales para datos estáticos

Características principales

- Acceso directo mediante índices numéricos
- Tamaño fijo predefinido en la declaración
- Memoria contigua y secuencial
- Eficiente para lectura aleatoria

Aplicaciones prácticas

- Listas de elementos con tamaño conocido
- Tablas de búsqueda estáticas
- Almacenamiento de datos secuenciales



Listas enlazadas: estructuras dinámicas

Ventajas clave

- Tamaño dinámico, crece según necesidad
- Inserción y eliminación eficientes
- No requiere memoria contigua
- Optimización para modificaciones frecuentes

Aplicaciones prácticas

- Implementación de pilas y colas
- Gestión de memoria dinámica
- Listas con tamaño variable desconocido

Pilas: estructura LIFO en acción



Control de procesos

Gestión de llamadas a funciones y recursión en programas



Operaciones reversibles

Implementación de función Deshacer en editores de texto



Navegación web

Historial de páginas visitadas y botón Atrás del navegador



Colas: estructura FIFO en gestión

Gestión de turnos

Sistemas de atención al cliente, colas de impresión, procesamiento de tareas en orden

Planificación de procesos

Sistemas operativos: gestión de procesos en ejecución y cola de listos

Buffering de datos

Transmisión de datos en redes, lectura/escritura de archivos

Matrices: datos bidimensionales

Características

- Estructura de dos dimensiones (filas x columnas)
- Acceso mediante dos índices (i, j)
- Representación tabular de información

Aplicaciones prácticas

- Hoja de cálculo y tablas de datos
- Imágenes digitales (píxeles en matriz)
- Juegos de tablero (ajedrez, sudoku)
- Sistemas de coordenadas y gráficos

MATARKIX			TAN	COK	SFEGINCIHE	
SKE	TXEK	RELM	CAMC	FRMK	PUDM	CSKGNPHKS
FI	10/20	8000	30	23	10	150
①	-15	-2	20	\$0	1	26
HC	8.00	810	4.0	\$2	31	\$5
MU	322	110	4.0	4.5	80	4.18
FFO	6300	130	8.0	425	30	\$3
INT	6230	190	4.0	\$35	40	4.2
FBEI	0201	140	5.0	330	50	\$23
ICK	2300	190	300	320	77	10.14
IOB	3320	120	8.0	\$20	6	4.15
IOD	6320	180	970	\$35	10	\$14
IBID	17300	100	010	433	30	\$70
FBB	0/630	190	6.0	\$73	26	\$76
P4	12030	100	3.0	\$75	30	\$73
ISSI	62800	130	8.0	\$73	30	\$36
8F30 A O	6.0	100	4.0	570	55	4.0
☑	☐	120	9.0	\$75	50	\$5
TI1	07230	120	\$0	\$70	50	\$7
ES	8.27	120	3.0	2.10	30	\$0

Errores comunes en la selección

Uso inadecuado de memoria

Arrays de tamaño excesivo o listas enlazadas innecesarias consumen recursos de forma ineficiente

Pérdida de eficiencia

Elegir estructuras con operaciones costosas para las tareas más frecuentes del problema

Ignorar el análisis previo

Seleccionar la primera estructura conocida sin evaluar las características del problema

Despreciar la escalabilidad

No considerar el crecimiento futuro del volumen de datos y operaciones

Conclusiones y buenas prácticas

01

Análisis exhaustivo del problema

Identificar requisitos de acceso, operaciones y volumen de datos

02

Selección consciente de estructura

Comparar opciones y elegir la que optimice rendimiento y memoria

03

Considerar casos extremos

Evaluar comportamiento con volumen mínimo y máximo de datos

04

Documentar la decisión

Registrar el razonamiento para mantenimiento futuro del software

La selección adecuada de estructuras de datos es una **competencia fundamental** en el desarrollo de software eficiente y escalable.